AgriPulse — Engineering Progress Report

AI Predictive Maintenance for Irrigation Pumps — Graduation Project, AI Engineering Program, Faculty of Engineering, Mansoura University

---

1. Project Overview

AgriPulse is a predictive-maintenance AI platform for centrifugal irrigation pumps: it learns normal pump behaviour from multi-sensor telemetry, detects early deviations, converts them into a calibrated failure-risk score with a 6-to-12-hour warning horizon, and generates a standards-grounded maintenance work order through a Retrieval-Augmented Generation (RAG) engine. The full scope and 67-task execution plan are documented in *docs/*; this report covers progress on the first workstream only (WS1 — data pipeline).

2. Work Completed — WS1 (Data Pipeline)

The data pipeline (tasks T08–T13 of the plan) has been implemented and run end to end against the real 220,320-row sensor dataset — not just designed, but working, tested code producing real output.

Task	Deliverable	Status
T08 — Ingest & profile dataset	profile_report.json	Done — verified
T09 — Clean data & sensor handling	sensor_handling.md	Done — policy corrected (see §3)
T10 — Labeling (WARNING / RUL targets)	labeling.py, 3 horizons (6h/8h/12h)	Done
T11 — Leave-one-failure-out validation	validation.py, 7 folds	Done
T12 — Feature engineering library	features.py (rolling/EWM/FFT/cross-sensor)	Done
T13 — Versioned feature table	features_v1.parquet, 1,053 columns	Done
T14–T20 — Model training (anomaly, risk, RUL)	—	In progress

Repository is organized by workstream (ml/, rag/, backend/, frontend/, iot/) and under git version control.

3. Key Finding During Data Cleaning

Two sensors had multi-day data gaps directly overlapping two failure events

The original plan called for forward-fill / linear interpolation on sensor_50 and sensor_51. Profiling the actual data showed this would have been a mistake: sensor_50 has a single ~77,000-minute (53-day) gap that swallows breakdown event #7 entirely, and sensor_51 has a ~15,356-minute (10.6-day) gap covering the full pre-failure warning window of event #5 — the largest failure in the dataset (140.5 hours of downtime).

Filling gaps that size would have fabricated weeks of fake sensor readings through the exact events the model is supposed to learn from. The pipeline now leaves long gaps as missing and adds a `_was_missing` indicator column instead, and only interpolates genuinely short gaps (recalibrated threshold: 30 minutes, based on the actual gap-length distribution in the data). This is documented in ml/reports/sensor_handling.md.

4. Validated Dataset Facts

All headline numbers quoted in the planning documents were checked against the real CSV and confirmed accurate:

Claim	Verified value
Total rows	220,320 (Apr 1 – Aug 31, 2018, 1-minute intervals, no gaps/duplicates)
Sensor channels	52 (sensor_00 .. sensor_51)
Target distribution	NORMAL 93.43% / RECOVERING 6.57% / BROKEN 0.003% (7 rows)
Breakdown events	7, timestamps match exactly to the minute
sensor_15	100% missing — dropped

5. Open Items Needing Supervisor / Team Input

- **Dataset provenance:** the data's structure (perfectly regular, zero gaps/duplicates, anonymized sensor_00..51 naming) matches a known public predictive-maintenance pump dataset rather than original Egyptian field telemetry. Should be cited properly in the thesis rather than implied as proprietary field data.
- **Team & supervisor fields:** the official university template still has placeholder names, credit hours, and GPA — needs to be filled before submission.
- **External accounts:** LLM API key (OpenAI/Gemini), a cloud database (TimescaleDB/Neon), and a Vercel account are needed once work reaches the RAG engine and backend/deployment stages (WS4, WS6, WS9) — not required yet.
- **Budget:** the ~$33 allocated for cloud/LLM tokens looks tight if development involves many iterations plus a live demo.

6. Next Steps

Proceed with T14–T20: train the baseline anomaly detectors (Isolation Forest, LSTM-Autoencoder), the supervised risk classifier (XGBoost/LightGBM), probability calibration and Green-to-Red risk levels, the persistence/hysteresis alert logic, and the RUL regressor — evaluated per failure event via the leave-one-failure-out folds already built, given only 7 total failure events are available.

Generated from the project repository's committed code and reports — every figure above is reproducible by re-running the referenced scripts.